1. 0/1 Knapsack Problem (Dynamic Programming)

```
def knapsack(weights, values, capacity):
    n = len(weights)
    dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]
    for i in range(1, n + 1):
        for w in range(capacity + 1):
            if weights[i - 1] <= w:
                dp[i][w] = max(
                    values[i - 1] + dp[i - 1][w - weights[i - 1]],
                    dp[i - 1][w]
                )
            else:
                dp[i][w] = dp[i - 1][w]
    return dp[n][capacity]
weights = [1, 3, 4, 5]
values = [1, 4, 5, 7]
capacity = 7

print("Maximum Profit:", knapsack(weights, values, capacity))
```

```
Maximum Profit: 9
```

2. Travelling Salesperson Problem (Dynamic Programming)

```
from functools import lru_cache
def tsp(graph):
    n = len(graph)
    @lru_cache(None)
    def visit(mask, pos):
        if mask == (1 << n) - 1:
            return graph[pos][0]
        ans = float('inf')
        for city in range(n):
            if mask & (1 << city) == 0:
                ans = min(
                    ans,
                    graph[pos][city] +
                    visit(mask | (1 << city), city)
                )
        return ans
    return visit(1, 0)
graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]
print("Minimum Cost:", tsp(graph))
```

```
Minimum Cost: 80
```

3. Optimal Binary Search Tree (OBST)

```
def optimal_bst(keys, freq):
    n = len(keys)
    cost = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        cost[i][i] = freq[i]
    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            cost[i][j] = float('inf')
            total_freq = sum(freq[i:j + 1])
            for r in range(i, j + 1):
                left = cost[i][r - 1] if r > i else 0
                right = cost[r + 1][j] if r < j else 0
                cost[i][j] = min(
                    cost[i][j],
                    left + right + total_freq
                )
    return cost[0][n - 1]
keys = [10, 20, 30]
freq = [34, 8, 50]
print("Optimal BST Cost:", optimal_bst(keys, freq))
```

```
Optimal BST Cost: 142
```

4. 0/1 Knapsack Problem

```
def knapsack(wt, val, W):
    n = len(wt)
    dp = [[0]*(W+1) for _ in range(n+1)]
    for i in range(1, n+1):
        for w in range(W+1):
            if wt[i-1] <= w:
                dp[i][w] = max(val[i-1] + dp[i-1][w-wt[i-1]],
                               dp[i-1][w])
            else:
                dp[i][w] = dp[i-1][w]
    return dp[n][W]
wt = [1, 3, 4, 5]
val = [1, 4, 5, 7]
W = 7
print("Maximum Profit =", knapsack(wt, val, W))
```

```
Maximum Profit = 9
```

5. 0/1 Knapsack Using Memoization

```
def knapsack(W, wt, val, n, memo):
    if n == 0 or W == 0:
        return 0
    if memo[n][W] != -1:
        return memo[n][W]
    if wt[n-1] <= W:
        memo[n][W] = max(
            val[n-1] + knapsack(W-wt[n-1], wt, val, n-1, memo),
            knapsack(W, wt, val, n-1, memo)
```

```
            )
        else:
            memo[n][W] = knapsack(W, wt, val, n-1, memo)
        return memo[n][W]
    wt = [1,3,4,5]
    val = [1,4,5,7]
    W = 7
    memo = [[-1]*(W+1) for _ in range(len(wt)+1)]
    print("Maximum Profit =", knapsack(W, wt, val, len(wt), memo))
```

```
Maximum Profit = 9
```

6. Subset Sum Problem

```
def subset_sum(arr, target):
    n = len(arr)
    dp = [[False]*(target+1) for _ in range(n+1)]
    for i in range(n+1):
        dp[i][0] = True
    for i in range(1, n+1):
        for j in range(1, target+1):
            if arr[i-1] <= j:
                dp[i][j] = dp[i-1][j] or dp[i-1][j-arr[i-1]]
            else:
                dp[i][j] = dp[i-1][j]
    return dp[n][target]
print(subset_sum([3,34,4,12,5,2], 9))
```

```
True
```

7. Travelling Salesperson Problem (DP Version)

```
from functools import lru_cache
graph = [
    [0,10,15,20],
    [10,0,35,25],
    [15,35,0,30],
    [20,25,30,0]
]
n = len(graph)
@lru_cache(None)
def tsp(mask, pos):
    if mask == (1<<n)-1:
        return graph[pos][0]
    ans = float('inf')
    for city in range(n):
        if not mask & (1<<city):
            ans = min(ans,
                      graph[pos][city] +
                      tsp(mask | (1<<city), city))
    return ans
print("Minimum Cost =", tsp(1,0))
```

```
Minimum Cost = 80
```

8. Travelling Salesperson Using DP Table

```
def tsp_dp(graph):
    n = len(graph)
    dp = [[float('inf')]*n for _ in range(1<<n)]
    dp[1][0] = 0
    for mask in range(1<<n):
        for u in range(n):
            if mask & (1<<u):
                for v in range(n):
                    if not mask & (1<<v):
                        dp[mask | (1<<v)][v] = min(
                            dp[mask | (1<<v)][v],
                            dp[mask][u] + graph[u][v]
                        )
    ans = float('inf')
    for i in range(1,n):
        ans = min(ans,
                  dp[(1<<n)-1][i] + graph[i][0])
    return ans
graph = [
    [0,10,15,20],
    [10,0,35,25],
    [15,35,0,30],
    [20,25,30,0]
]
print("Minimum Cost =", tsp_dp(graph))
```

```
Minimum Cost = 80
```

9. Optimal Binary Search Tree (OBST)

```
def optimal_bst(freq):
    n = len(freq)
    cost = [[0]*n for _ in range(n)]
    for i in range(n):
        cost[i][i] = freq[i]
    for L in range(2, n+1):
        for i in range(n-L+1):
            j = i+L-1
            cost[i][j] = float('inf')
            total = sum(freq[i:j+1])
            for r in range(i, j+1):
                left = cost[i][r-1] if r > i else 0
                right = cost[r+1][j] if r < j else 0
                cost[i][j] = min(cost[i][j],
                                 left + right + total)
    return cost[0][n-1]
print("Optimal Cost =", optimal_bst([34,8,50]))
```

```
Optimal Cost = 142
```

10. Matrix Chain Multiplication

```
def matrix_chain(arr):
    n = len(arr)
    dp = [[0]*n for _ in range(n)]
    for L in range(2, n):
        for i in range(1, n-L+1):
            j = i+L-1
            dp[i][j] = float('inf')
            for k in range(i, j):
                q = dp[i][k] + dp[k+1][j] + arr[i-1]*arr[k]*arr[j]
                dp[i][j] = min(dp[i][j], q)
    return dp[1][n-1]
print("Minimum Multiplications =", matrix_chain([10,20,30,40,30]))
```

```
Minimum Multiplications = 30000
```

11. Coin Change (Minimum Coins)

```
def min_coins(coins, amount):
    dp = [float('inf')] * (amount+1)
    dp[0] = 0
    for i in range(1, amount+1):
        for c in coins:
            if c <= i:
                dp[i] = min(dp[i], dp[i-c]+1)
    return dp[amount]
print("Minimum Coins =", min_coins([1,2,5], 11))
```

```
Minimum Coins = 3
```

12. Rod Cutting Problem

```
def rod_cut(price, n):
    dp = [0]*(n+1)
    for i in range(1, n+1):
        for j in range(i):
            dp[i] = max(dp[i],
                        price[j] + dp[i-j-1])
    return dp[n]
print("Maximum Revenue =", rod_cut([1,5,8,9,10,17,17,20], 8))
```

```
Maximum Revenue = 22
```

13. Minimum Cost Path

Gemini

```
def min_cost(cost):
    m = len(cost)
    n = len(cost[0])
    dp = [[0]*n for _ in range(m)]
    dp[0][0] = cost[0][0]
    for i in range(1,m):
        dp[i][0] = dp[i-1][0] + cost[i][0]
    for j in range(1,n):
        dp[0][j] = dp[0][j-1] + cost[0][j]
```

```
    for i in range(1,m):
        for j in range(1,n):
            dp[i][j] = cost[i][j] + min(dp[i-1][j],
                                        dp[i][j-1])
    return dp[m-1][n-1]
cost = [
    [1,3,1],
    [1,5,1],
    [4,2,1]
]
print("Minimum Cost =", min_cost(cost))
```

```
Minimum Cost = 7
```